1. 8-Puzzle Problem

Aim:

To write a Python program to solve the 8-puzzle problem using A* algorithm.

Algorithm:

1. Define the goal state and the initial state.
2. Create a priority queue (open list) to store the nodes to be explored, prioritized by their heuristic cost (Manhattan distance + cost to reach the node).
3. Push the initial state into the open list.
4. Loop until the open list is empty:
   a. Pop the node with the lowest cost.
   b. If it is the goal state, we are done.
   c. Otherwise, generate all possible moves (up, down, left, right).
   d. For each valid move, calculate the new state and its cost.
   e. Push the new states into the open list if they haven't been visited.
5. Print the steps to reach the goal.

Program:

```
import copy import
heapq

goal_state = [[1, 2, 3], [4, 5, 6], [7, 8, 0]]

def find_blank(state): for i in
    range(3):
        for j in range(3):
            if state[i][j] == 0: return i, j

def get_moves(state):
    i, j = find_blank(state) moves = []
    if i > 0: moves.append((-1, 0)) # up if i < 2:
    moves.append((1, 0))                    # down
    if j > 0: moves.append((0, -1)) # left
    if j < 2: moves.append((0, 1))          # right
    return moves

def manhattan(state): dist = 0
    for i in range(3):
        for j in range(3): val =
            state[i][j] if val != 0:
                gi, gj = (val - 1) // 3, (val - 1) % 3 dist += abs(gi - i) + abs(gj
                - j)
    return dist
```

```
def solve_8_puzzle(start_state): pq = []
    heapq.heappush(pq, (manhattan(start_state), 0, start_state, [])) visited = set()

    while pq:
        f, g, state, path = heapq.heappop(pq)

        state_tuple = tuple(tuple(row) for row in state) if state_tuple in visited:
            continue
        visited.add(state_tuple)

        if state == goal_state: return path +
            [state]

        i, j = find_blank(state)
        for di, dj in get_moves(state): new_state =
            copy.deepcopy(state)
            new_state[i][j], new_state[i+di][j+dj] = new_state[i+di][j+dj], new_state[i][j]

            if tuple(tuple(row) for row in new_state) not in visited:
                    heapq.heappush(pq, (g + 1 + manhattan(new_state), g + 1, new_state, path +
[state]))
    return None

start = [[1, 2, 3], [4, 0, 6], [7, 5, 8]]
print("Initial State:")
for row in start: print(row) print("Solving...")
path = solve_8_puzzle(start) if path:
    print(f"Goal reached in {len(path)-1} steps.") for step, state in
    enumerate(path):
        print(f"Step {step}:")
        for row in state: print(row) print()
else:
    print("No solution found.")
```

Output:

```
Initial State:
 [1,   2,   3]
 [4,   0,   6]
 [7,   5,   8]
Solving...
Goal reached in 2 steps.
Step 0:
 [1,   2,   3]
 [4,   0,   6]
 [7,   5,   8]
```

Step 1:

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

Step 2:

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

2. 8-Queen Problem

Aim:

To write a Python program to solve the 8-Queen problem using backtracking.

Algorithm:

1. Start in the leftmost column.
2. If all queens are placed, return true.
3. Try all rows in the current column. Do the following for every tried row:
   a. If the queen can be placed safely in this row, mark this [row, column] as part of the solution.
   b. Recursively check if placing queen here leads to a solution.
   c. If it leads to a solution, return true.
   d. If it doesn't, unmark this [row, column] (Backtrack) and go to step (a) to try other rows.
4. If all rows have been tried and nothing worked, return false to trigger backtracking.

Program:

```
def is_safe(board, row, col):
    # Check this row on left side for i in
    range(col):
        if board[row][i] == 1: return False

    # Check upper diagonal on left side
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)): if board[i][j] == 1:
            return False

    # Check lower diagonal on left side
    for i, j in zip(range(row, 8, 1), range(col, -1, -1)): if board[i][j] == 1:
            return False

    return True

def solve_nq_util(board, col): if col >= 8:
        return True

    for i in range(8):
        if is_safe(board, i, col): board[i][col] = 1
            if solve_nq_util(board, col + 1): return True
            board[i][col] = 0 # Backtrack return False

def solve_8_queen():
    board = [[0 for _ in range(8)] for _ in range(8)]
```

```
    if not solve_nq_util(board, 0): print("Solution does not
        exist") return False

    print("One possible arrangement:") for row in board:
        print(" ".join("Q" if x else "." for x in row)) return True

solve_8_queen()
```

Output:

One possible arrangement:

```
Q . . . . . .
. . . . Q . . .
. . . . . . . .Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

3. Water Jug Problem

Aim:

To write a Python program to solve the Water Jug problem using Breadth-First Search (BFS).

Algorithm:

1. Initialize two jugs, Jug1 with capacity X and Jug2 with capacity Y. Target is to measure Z liters.
2. Start with initial state (0, 0).
3. Use a queue (for BFS) and keep track of visited states.
4. Generate next possible states:
   - Fill Jug1 completely
   - Fill Jug2 completely
   - Empty Jug1 completely
   - Empty Jug2 completely
   - Pour Jug1 to Jug2 until Jug2 is full or Jug1 is empty
   - Pour Jug2 to Jug1 until Jug1 is full or Jug2 is empty
5. If target is reached in either jug, stop and trace the path.
6. Print the steps.

Program:

```
from collections import deque

def solve_water_jug(capacity1, capacity2, target): visited = set()
    queue = deque([((0, 0), [])])

    while queue:
        (jug1, jug2), path = queue.popleft()

        if (jug1, jug2) in visited: continue
        visited.add((jug1, jug2))

        if jug1 == target or jug2 == target: path.append((jug1, jug2))
            return path

        states = [
            (capacity1, jug2), # Fill jug 1 (jug1, capacity2),
            # Fill jug 2 (0, jug2),          # Empty jug 1
            (jug1, 0),                       # Empty jug 2 #
            Pour jug1 to jug2
            (jug1 - min(jug1, capacity2 - jug2), jug2 + min(jug1, capacity2 - jug2)), # Pour jug2 to jug1
            (jug1 + min(jug2, capacity1 - jug1), jug2 - min(jug2, capacity1 - jug1))
        ]
```

```
            for state in states:
                if state not in visited:
                    queue.append((state, path + [(jug1, jug2)]))

    return None

cap1, cap2, target = 4, 3, 2
print(f"Jugs: {cap1}L, {cap2}L. Target: {target}L") path =
solve_water_jug(cap1, cap2, target)

if path:
    print("Steps to solve:") for step in
    path:
        print(f"Jug1: {step[0]}L, Jug2: {step[1]}L")
else:
    print("No solution.")
```

Output:

```
Jugs: 4L, 3L. Target: 2L Steps to
solve:
Jug1: 0L, Jug2: 0L
Jug1: 4L, Jug2: 0L
Jug1: 1L, Jug2: 3L
Jug1: 1L, Jug2: 0L
Jug1: 0L, Jug2: 1L
Jug1: 4L, Jug2: 1L
Jug1: 2L, Jug2: 3L
```

4. Crypt-Arithmetic Problem

Aim:

To write a Python program to solve the Crypt-Arithmetic problem (e.g., SEND + MORE = MONEY).

Algorithm:

1. Extract all unique letters from the words.
2. If there are more than 10 unique letters, return false (no solution).
3. Use backtracking to assign a unique digit (0-9) to each letter.
4. For each assignment, check if the leading letters are non-zero.
5. Substitute the assigned digits into the words and check if the equation (Word1 + Word2 == Result) holds.
6. If it holds, print the mapping.
7. Otherwise, backtrack and try different assignments.

Program:

```
import itertools

def solve_cryptarithmetic(word1, word2, result): letters = set(word1 + word2
    + result)
    if len(letters) > 10: return None

    letters = list(letters)
    first_letters = {word1[0], word2[0], result[0]}

    # Try all combinations of digits for the unique letters
    for perm in itertools.permutations(range(10), len(letters)): mapping = dict(zip(letters, perm))

        # Leading digits cannot be zero
        if any(mapping[char] == 0 for char in first_letters): continue

        # Evaluate the words
        num1 = int("".join(str(mapping[char]) for char in word1)) num2 =
        int("".join(str(mapping[char]) for char in word2)) res_num = int("".join(str(mapping[char])
        for char in result))

        if num1 + num2 == res_num:
            return mapping, num1, num2, res_num return None

w1, w2, res = "SEND", "MORE", "MONEY"
print(f"Solving: {w1} + {w2} = {res}") solution =
solve_cryptarithmetic(w1, w2, res)

if solution:
    mapping, num1, num2, res_num = solution
```

```
print("Solution found:") print(mapping)
print(f"            {num1}")
print(f"+ {num2}")
print(f"                    ")
print(f" {res_num}") else:
print("No solution exists.")
```

Output:

```
Solving: SEND + MORE = MONEY
Solution found:
{'M': 1, 'R': 8, 'O': 0, 'D': 7, 'N': 6, 'E': 5, 'S': 9, 'Y': 2}
  9567
+ 1085
------
 10652
```

5. Missionaries Cannibal problem

Aim:

To write a Python program to solve the Missionaries and Cannibals problem using state space search.

Algorithm:

1. Define the initial state (3 missionaries, 3 cannibals, boat on the left side) and the goal state (0, 0, right side).
2. Define the constraints: at any time on either side, missionaries must not be outnumbered by cannibals.
3. Boat can carry 1 or 2 people.
4. Use Breadth-First Search (BFS) to explore the state space.
5. Generate next valid states by moving 1 or 2 valid people across the river.
6. Keep track of visited states to avoid cycles.
7. Once the goal state is reached, print the path from initial state to goal state.

Program:

```
from collections import deque

def is_valid(m, c):
    # Missionaries cannot be outnumbered by cannibals on either side if m < 0 or c < 0 or m > 3 or
    c > 3: return False
    if (m > 0 and m < c) or (3-m > 0 and 3-m < 3-c): return False return True

def get_next_states(m, c, boat): states = []
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
    for dm, dc in moves:
        if boat == 1: # Boat on left, moving to right new_m, new_c = m - dm, c
            - dc
            if is_valid(new_m, new_c): states.append((new_m, new_c, 0))
        else:  # Boat on right, moving to left new_m, new_c = m + dm, c +
            dc
            if is_valid(new_m, new_c): states.append((new_m, new_c, 1))
    return states

def solve_mc():
    # State: (missionaries_left, cannibals_left, boat_pos) start = (3, 3, 1)
    goal = (0, 0, 0)

    queue = deque([(start, [])]) visited = set()

    while queue:
        (m, c, boat), path = queue.popleft()
```

```
            if (m, c, boat) in visited: continue
            visited.add((m, c, boat))

            if (m, c, boat) == goal:
                return path + [(m, c, boat)]

            for next_state in get_next_states(m, c, boat): if next_state not in
                visited:
                    queue.append((next_state, path + [(m, c, boat)]))
    return None

path = solve_mc() if path:
    print("Missionaries(M) & Cannibals(C) on Left Side. Boat(1=Left, 0=Right)") for step, (m, c, boat) in
    enumerate(path):
        print(f"Step {step}: {m}M, {c}C (Left), Boat={boat}")
else:
    print("No solution found.")
```

Output:

Missionaries(M) & Cannibals(C) on Left Side. Boat(1=Left, 0=Right) Step 0: 3M, 3C (Left), Boat=1
Step 1: 3M, 1C (Left), Boat=0 Step 2: 3M,
2C (Left), Boat=1 Step 3: 3M, 0C (Left),
Boat=0 Step 4: 3M, 1C (Left), Boat=1 Step
5: 1M, 1C (Left), Boat=0 Step 6: 2M, 2C
(Left), Boat=1 Step 7: 0M, 2C (Left),
Boat=0 Step 8: 0M, 3C (Left), Boat=1 Step
9: 0M, 1C (Left), Boat=0 Step 10: 1M, 1C
(Left), Boat=1 Step 11: 0M, 0C (Left),
Boat=0